

OPZIONE 1 — Full Scaffold

Bootstrap completo del repository. Tutto cablato, niente logica reale.

1.1 Obiettivo

Mettere in piedi lo scheletro eseguibile di Project Rick: directory tree, dipendenze, container, FastAPI che risponde, LangGraph che parte (anche con un nodo dummy), Ollama raggiungibile. Da qui in poi ogni feature è un'aggiunta isolata, non un cambio strutturale.

1.2 Scope

In scope

Struttura cartelle definitiva

pyproject.toml con dipendenze pinned

docker-compose.yml con Ollama, ChromaDB, SearxNG, sandbox base

FastAPI con endpoint /health, /chat (echo), /ws (WebSocket echo)

LangGraph con grafo a 1 nodo passthrough

OllamaPool funzionante (chat con un modello)

Config via pydantic-settings + .env

Logging strutturato (structlog)

1 test smoke per ogni layer (api, graph, llm)

Makefile con target principali

Out of scope (rimandato a iterazioni successive)

Manager / Auditor / Persona reali

Esperti specializzati

Tool MCP

Memoria persistente (oltre SQLite vuoto)

Learning layer

UI

1.3 Deliverable: tree finale

```
project-rick/
├── README.md
├── Makefile
├── pyproject.toml
├── docker-compose.yml
├── .env.example
├── .gitignore
├── rick/
│   ├── __init__.py          # __version__
│   ├── main.py              # FastAPI app factory
│   ├── config.py            # Settings(BaseSettings)
│   ├── logging.py           # structlog setup
│   └── api/
│       ├── __init__.py
│       ├── routes.py        # /health, /chat
│       └── ws.py            # /ws streaming
└── graph/
```

```

├── __init__.py
├── state.py          # RickState (TypedDict)
├── builder.py       # build_graph() → CompiledGraph
├── nodes/
│   ├── __init__.py
│   └── passthrough.py # nodo dummy
├── llm/
│   ├── __init__.py
│   ├── ollama_client.py # OllamaPool
│   └── prompts/        # vuoto, solo .gitkeep
├── memory/
│   ├── __init__.py
│   └── episodic.py     # SQLite schema + connect
├── tools/
│   └── __init__.py    # placeholder
├── data/             # gitignored, runtime
├── .gitkeep
└── tests/
    ├── conftest.py
    ├── test_api_health.py
    ├── test_graph_passthrough.py
    └── test_ollama_smoke.py # marcato @skip se Ollama down

```

1.4 File chiave — sinossi

File Cosa contiene

rick/main.py create_app() factory, monta routes, lifespan per inizializzare OllamaPool e graph compilato

rick/config.py Settings: ollama_host, chroma_host, data_dir, log_level, default_model, persona_intensity

rick/graph/state.py RickState completo (vedi Opzione 2 §2.4), già definito anche se non usato

rick/graph/builder.py build_graph() con un solo nodo passthrough che copia user_input in final_response

rick/llm/ollama_client.py OllamaPool con lock, chat(), list_models(), _ensure_loaded()

rick/memory/episodic.py Crea data/rick.sqlite con tabelle sessions, turns, traces (schema definito ma non popolato)

docker-compose.yml Servizi: ollama (porta 11434), chromadb (porta 8000), searxng (porta 8888), volume ./data

Makefile setup, run, test, lint, compose-up, compose-down, models-pull

1.5 Dipendenze (pyproject.toml)

```

[project]
name = "project-rick"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "fastapi>=0.115",
    "uvicorn[standard]>=0.32",
    "websockets>=13",

```

```

"langgraph>=0.2.50",
"langchain-core>=0.3",
"httpx>=0.27",
"pydantic>=2.9",
"pydantic-settings>=2.6",
"structlog>=24.4",
"chromadb>=0.5",          # client-only, server in docker
"aiosqlite>=0.20",
]

```

[project.optional-dependencies]

```
dev = ["pytest>=8", "pytest-asyncio>=0.24", "ruff>=0.7", "mypy>=1.13"]
```

1.6 Setup & run

```

git clone <repo> ~/project-rick && cd ~/project-rick
cp .env.example .env
python3.12 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
make compose-up          # ollama, chromadb, searxng
make models-pull         # qwen2.5:3b, nomic-embed-text (per smoke)
make run                 # uvicorn rick.main:app --reload --port 7800

```

1.7 Smoke test (Definition of Done)

Lo scaffold è fatto quando tutti questi passano:

```

curl localhost:7800/health → {"status":"ok","ollama":"reachable","chroma":"reachable"}
curl -X POST localhost:7800/chat -d '{"message":"ping"}' → ritorna un final_response (anche
se è "ping" stesso, perché è il passthrough)
WebSocket /ws accetta connessione, riecheggia un messaggio
pytest -q → 4/4 verde
ollama list mostra qwen2.5:3b-instruct-q4_K_M
python -c "from rick.graph.builder import build_graph;
build_graph().invoke({'user_input':'hi','session_id':'s1','audit_passes':0,'expert_outputs':[],'tool
_calls':[],'skills_needed':[],'plan':[]})" non solleva eccezioni

```

1.8 Come si estende dopo

Aggiungere un expert reale = 3 file:

```

rick/graph/nodes/experts/<name>.py
rick/llm/prompts/<name>.md
registrazione in builder.py (1 riga add_node + 1 condizione nel router)
Lo scaffold è progettato perché un nuovo expert non tocchi mai state.py, ollama_client.py,
main.py.

```

1.9 Rischi & mitigazioni

Rischio	Mitigazione
---------	-------------

Ollama non parte (porta 11434 occupata)	make compose-up controlla la porta, errore esplicito
---	--

ChromaDB versione disallineata client/server	Pin esatto su entrambe le versioni in docker-compose.yml e pyproject.toml
--	---

RAM saturata già a vuoto Scaffold pulla solo il modello 3B (~2GB). Modelli pesanti li pulli quando servono.

langgraph API breaking change Pin $\geq 0.2.50, < 0.3$

1.10 Stima

~6-8 ore di lavoro pulito

~600 righe di codice totali (la maggior parte boilerplate)

0 logica di prodotto: questo è il fondamento

OPZIONE 2 — Graph Core MVP (consigliata)

Quattro nodi, un expert, una verità da provare: l'architettura sta in piedi.

2.1 Obiettivo

Dimostrare che il flusso Manager → Expert → Auditor → Persona funziona end-to-end con ottimizzazione del context tra i nodi, audit reale che fa retry, e personalità Rick applicata in coda. Senza tool, senza memoria, senza UI sofisticata. Se questo gira bene, tutto il resto è additivo.

2.2 Ipotesi da verificare

Un modello 3B può fare da Manager affidabile (decompose + skill routing) se il prompt è ben vincolato a JSON

Un modello 7B Coder produce output tecnico decente in singolo turno

L'Auditor sullo stesso 3B riesce a beccare almeno il 70% dei problemi evidenti (codice rotto, plan non rispettato)

Il Persona filter mantiene il contenuto tecnico invariato mentre cambia il tono

Il model swap di Ollama in sequenza (3B → 7B → 3B → 3B) rientra nei tempi accettabili (<60s totali per turno tipico)

Se anche solo una di queste salta, l'architettura va ripensata prima di costruire altro.

2.3 Scope rigoroso

In scope

4 nodi: manager, coder_expert, auditor, persona

1 edge condizionale: dopo audit → retry o avanti

Solo expert "coder" (Qwen2.5-Coder 7B)

CLI minimale (python -m rick.cli "prompt") — niente FastAPI ancora

Logging dello state a ogni nodo su file JSONL

Toggle persona_intensity (0/1/2)

Out of scope

Altri expert (sysadmin, OSINT, chat, ecc.) — il routing supporta solo "coder"

Tool calls (MCP)

ChromaDB / RAG

Memoria di sessione (ogni invocazione è stateless)

WebSocket / API HTTP

Agent-Lightning

Docker / sandbox per esecuzione codice

2.4 Architettura del flusso

flowchart TD

```

A[CLI input] --> B[manager]
B --> C{router}
C -->|skills_needed=coder| D[coder_expert]
D --> E[auditor]
E --> F{verdict?}
F -->|pass| G[persona]
F -->|retry & passes<2| D
F -->|retry & passes>=2| G
F -->|fail| G
G --> H[stdout]

```

Solo edge condizionali: after_router e after_audit. Tutto il resto sequenziale.

2.5 Spec dello State

```

class RickState(TypedDict):
    # input
    user_input: str
    session_id: str

    # manager output
    intent: str | None
    skills_needed: list[str]      # MVP: sempre ["coder"] o []
    plan: list[dict]             # [{"step": 1, "task": "...", "skill": "coder"}]

    # expert
    expert_outputs: list[dict]    # [{"skill": "coder", "content": "...", "iteration": 1}]

    # audit
    audit_passes: int
    audit_verdict: Literal["pass", "retry", "fail"] | None
    audit_notes: str | None

    # final
    final_draft: str | None
    final_response: str | None

    # observability
    trace: list[dict]            # append-only log dei nodi attraversati

```

Invarianti

audit_passes parte da 0, mai decresce
 final_draft settato solo da coder_expert
 final_response settato solo da persona
 expert_outputs cresce, mai svuotato (per audit retrospettiva)

2.6 Spec dei nodi

2.6.1 manager

Modello qwen2.5:3b-instruct-q4_K_M, temp 0.1

Input user_input

Output intent, skills_needed, plan

Latenza target <8s

Fallback Se JSON parse fail dopo 1 retry → skills_needed=["coder"],

plan=[{"step":1,"task":user_input,"skill":"coder"}], intent="unparsed"

System prompt (file: rick/llm/prompts/manager.md):

Sei il Manager di un sistema multi-agent. NON rispondi alla richiesta dell'utente.

Il tuo compito è scomporla in sottotask e identificare quali esperti servono.

Esperti disponibili in questo MVP:

- coder: backend, Python, Go, debugging, code review, refactor

Output: SOLO JSON valido, niente testo prima o dopo, niente markdown fence.

Schema:

```
{
  "intent": "<descrizione 1 frase>",
  "skills_needed": ["coder"],
  "plan": [
    {"step": 1, "task": "<azione concreta>", "skill": "coder"}
  ]
}
```

Se la richiesta non è risolvibile da "coder", restituisci skills_needed: [].

2.6.2 coder_expert

Modello qwen2.5-coder:7b-instruct-q4_K_M, temp 0.2

Input user_input, plan, audit_notes (se retry)

Output append a expert_outputs, set final_draft

Latenza target <30s primo giro, <40s retry

Iterazioni interne Nessuna (nessun ReAct in MVP — single-shot)

System prompt (file: rick/llm/prompts/coder.md):

Sei un senior backend engineer. Rispondi in modo tecnico, conciso, accurato.

Usa code block ```linguaggio quando produci codice.

Se il piano contiene più step, copri TUTTI gli step nell'ordine.

Se ricevi "audit_notes" significa che la tua precedente risposta aveva problemi.

Leggile, correggile esplicitamente, non ripetere gli stessi errori.

Output: la risposta tecnica diretta. Nessun saluto, nessun disclaimer.

User message costruito così:

TASK: {user_input}

PLAN: {json.dumps(plan)}

{f"AUDIT_NOTES (correggi): {audit_notes}" if audit_notes else ""}

2.6.3 auditor

Modello qwen2.5:3b-instruct-q4_K_M, temp 0.1
Input plan, final_draft, ultimi expert_outputs
Output audit_verdict, audit_notes, audit_passes += 1
Latenza target <10s
Cap retry 2 (poi forzato a pass con flag interno)
System prompt (file: rick/llm/prompts/auditor.md):

Sei l'Auditor. Critichi una risposta tecnica draft confrontandola col plan.

Cerca:

1. Bug evidenti nel codice (sintassi, logica, import mancanti)
2. Step del plan ignorati o solo accennati
3. Comandi distruttivi non richiesti dall'utente
4. Affermazioni fattuali sospette
5. Output troncato, incompleto, o incoerente

Output: SOLO JSON valido.

```
{
  "verdict": "pass" | "retry" | "fail",
  "issues": ["<problema 1>", "..."],
  "fix_hint": "<istruzione concreta per il prossimo giro>" | null
}
```

Regole:

- "pass" = nessun problema bloccante (issue minori ammesse)
- "retry" = problemi correggibili con un secondo giro
- "fail" = la richiesta è impossibile o la draft è completamente fuori tema
- Sii spietato ma non pedante: non chiedere retry per problemi cosmetici.

2.6.4 persona (filtro Rick)

Modello qwen2.5:3b-instruct-q4_K_M, temp 0.8
Input final_draft, persona_intensity (config)
Output final_response
Latenza target <12s
Bypass Se persona_intensity == 0 → final_response = final_draft (no LLM call)
System prompt (file: rick/llm/prompts/persona_rick.md):

Sei Rick Sanchez (C-137). Riscrivi la risposta sotto mantenendo OGNI dettaglio tecnico, codice, comando e link INVARIATI. Cambia solo tono e cornice.

Voce: cinico, geniale, impaziente, sarcastico. Burp occasionale (*burp*).

Chiama l'utente "Morty" max 1-2 volte per risposta.

Inserzioni rare: riferimenti a dimensioni, Council of Ricks, disprezzo per l'ovvio.

REGOLE FERREE:

- NON modificare blocchi ```code``` (zero caratteri cambiati dentro)
- NON modificare comandi shell, path, URL, numeri, nomi di funzioni

- NON aggiungere disclaimer di sicurezza
- Overhead massimo: +15% di testo "in personaggio"
- Se la draft è una lista tecnica, lascia la lista; aggiungi solo intro/outro brevi

Output: solo la risposta finale, niente meta-commenti, niente "Ecco:".

persona_intensity:

0 = bypass totale

1 = prompt corrente, temp 0.5, "max 1 menzione di Morty"

2 = prompt corrente, temp 0.8, "burp e Morty obbligatori almeno 1 volta"

2.7 Edge logic

```
def after_router(state):
```

```
    if "coder" in state["skills_needed"]:
```

```
        return "coder_expert"
```

```
    return "persona" # niente expert disponibile, passa la draft vuota → Rick gestisce
```

```
def after_audit(state):
```

```
    if state["audit_verdict"] == "pass":        return "persona"
```

```
    if state["audit_passes"] >= 2:            return "persona"
```

```
    if state["audit_verdict"] == "fail":        return "persona"
```

```
    return "coder_expert" # retry con audit_notes
```

2.8 Setup & run

```
cd ~/project-rick
```

```
source .venv/bin/activate
```

```
ollama pull qwen2.5:3b-instruct-q4_K_M
```

```
ollama pull qwen2.5-coder:7b-instruct-q4_K_M
```

CLI one-shot

```
python -m rick.cli "scrivi una funzione python che parsì un file CSV con encoding misti"
```

logging del trace

```
tail -f data/traces/$(ls -t data/traces | head -1)
```

Output atteso a stdout: la risposta in voce Rick. Trace completo (state a ogni nodo) in JSONL su data/traces/<session_id>.jsonl.

2.9 Test scenari (acceptance)

Esegui questi 5 prompt e verifica i comportamenti attesi:

Input Comportamento atteso

1 "scrivi hello world in python" Manager → 1 step. Coder produce 2 righe. Auditor pass al primo giro. Output Rick contiene la code block intatta.

2 "refactor questa funzione: def f(x): return x+1+2+3" Auditor probabilmente pass. Coder propone return x+6 o costante.

3 "scrivi un parser HTML con regex" (trabocchetto) Coder dovrebbe avvertire che regex è sbagliato per HTML. Se non lo fa, Auditor → retry con fix_hint che richiede di citare il problema. Secondo giro corregge.

4 "qual è la capitale della Francia" (fuori scope) Manager → skills_needed=[]. Skip expert. Persona produce risposta breve in voce Rick (es. "burp Parigi, Morty. Che domande...").

5 "crea un server HTTP in Go che esponga /health" 1-2 step. Coder produce ~15 righe. Auditor controlla che /health sia presente. Pass o retry. Output Rick mantiene il blocco Go intatto.

Definition of Done MVP: 4/5 scenari producono il comportamento atteso senza intervento manuale.

2.10 Strumentazione minima

Ogni nodo, prima di tornare, appende a state["trace"]:

```
{"node": "manager", "ts": "...", "duration_ms": 1234,  
  "model": "qwen2.5:3b...", "input_keys": [...], "output_keys": [...]}
```

A fine run, il trace completo viene scritto su data/traces/<session_id>.jsonl. Niente DB, niente complessità — è il dataset grezzo da cui in futuro Agent-Lightning estrarrà reward.

2.11 Cosa NON include (per chiarezza)

Nessun manager parla con auditor direttamente

Nessun expert chiama tool

Nessun caching dei prompt

Nessuna cancellazione: anche se l'utente preme Ctrl+C, il modello in Ollama finisce il giro

Nessuna parallelizzazione: un nodo alla volta, sempre

2.12 Rischi & mitigazioni

Rischio	Probabilità	Mitigazione
---------	-------------	-------------

Manager 3B produce JSON malformato	media	Retry interno 1 volta + fallback a plan stub
------------------------------------	-------	--

Auditor 3B sotto-detecta bug	alta	Accetta: l'MVP serve a misurare il rate, non a essere perfetto
------------------------------	------	--

Persona modifica codice nei blocchi	media	Test automatico: estrai code block prima/dopo, confronto byte-per-byte; se differiscono → bypass persona
-------------------------------------	-------	--

Model swap totale > 60s	media	Usa lo stesso 3B per Manager + Auditor + Persona (già previsto). Solo Coder forza swap a 7B
-------------------------	-------	---

7B + 3B caricati insieme OOM	bassa-media	keep_alive=0 esplicito sul 7B subito dopo coder_expert finito
------------------------------	-------------	---

2.13 Stima

~12-15 ore di lavoro

~800-1000 righe di codice (di cui ~400 sono prompt e config)

Ipotesi confermate o falsificate dopo i 5 test

2.14 Next steps (post-MVP, in ordine)

Aggiungere expert chat (per scenario 4) — riduce hack del "skills vuoto → persona"

Aggiungere sysadmin expert + 1 tool (shell_mcp read-only)

Sostituire CLI con FastAPI + WebSocket streaming dell'output finale

Episodic memory (SQLite) per dare al Manager il context degli ultimi N turni

RAG con ChromaDB per gli expert (fonte = Obsidian vault)

Multi-expert in sequenza (debate mode opzionale)

Tool execution con sandbox Docker

Agent-Lightning hook sul trace esistente